

Teach Module 18: Mockito for Test Isolation

This README captures the Module 18 teaching session, practice exercises, and lab. The bootcamp document was used only to identify the Module 18 topic list; the teaching content below is original explanatory material.

Module 18 Topics

Module 18 focuses on Mockito and isolated unit testing:

- Mock creation
- Stubbing behavior
- Verification patterns
- Test doubles
- Avoiding over-mocking
- Using AI assistance to generate mock setup and stubs
- Lab practice

1. What Mockito Is For

Mockito is a Java testing library used to replace real dependencies with fake ones during unit tests.

Suppose a service depends on a payment gateway:

```
public class OrderService {
    private final PaymentGateway paymentGateway;

    public OrderService(PaymentGateway paymentGateway) {
        this.paymentGateway = paymentGateway;
    }

    public boolean placeOrder(Order order) {
        if (order.total() <= 0) {
            return false;
        }

        return paymentGateway.charge(order.total());
    }
}
```

In a real system, `PaymentGateway` might call a bank, Stripe, PayPal, or another service. In a unit test, we do not want that external call. We only want to test `OrderService`.

So we replace the real dependency with a mock:

```
import static org.junit.jupiter.api.Assertions.assertTrue;
import static org.mockito.Mockito.mock;
import static org.mockito.Mockito.when;

import org.junit.jupiter.api.Test;

class OrderServiceTest {
```

```

@Test
void placeOrderReturnsTrueWhenPaymentSucceeds() {
    PaymentGateway paymentGateway = mock(PaymentGateway.class);

    when(paymentGateway.charge(100.0)).thenReturn(true);

    OrderService orderService = new OrderService(paymentGateway);
    Order order = new Order(100.0);

    boolean result = orderService.placeOrder(order);

    assertTrue(result);
}
}

```

The key line is:

```
PaymentGateway paymentGateway = mock(PaymentGateway.class);
```

This creates a fake `PaymentGateway` .

Then this line controls its behavior:

```
when(paymentGateway.charge(100.0)).thenReturn(true);
```

That is called stubbing. It means: when `charge(100.0)` is called, return `true` .

2. The Core Mockito Pattern

Most Mockito tests follow this shape:

```

Arrange: create mocks and define behavior
Act: call the real class under test
Assert: check the result
Verify: check important dependency interactions

```

The mental model:

```

Class under test = real
Dependencies = fake
Inputs = real
Assertions = check result or interaction

```

For example:

```

Real: OrderService
Fake: PaymentGateway

```

```
Real: Order
Check: Did placeOrder return true?
```

3. Mock Creation

A mock is a fake object used in place of a real dependency.

```
UserRepository userRepository = mock(UserRepository.class);
```

You usually mock dependencies such as:

- Repository classes
- HTTP clients
- Payment gateways
- Email senders
- External API clients
- Message queue publishers
- File storage clients

You usually do not mock simple data objects such as:

- User
- Order
- Product
- Address

Prefer real objects for simple domain models:

```
User user = new User("Alex", "alex@example.com");
Order order = new Order(100.0);
```

4. Stubbing Behavior

Stubbing defines what the mock should return.

```
when(userRepository.findById(1L)).thenReturn(Optional.of(user));
```

Example:

```
@Test
void returnsUserNameWhenUserExists() {
    UserRepository repository = mock(UserRepository.class);
    User user = new User(1L, "Asha");

    when(repository.findById(1L)).thenReturn(Optional.of(user));

    UserService service = new UserService(repository);

    String name = service.getUserName(1L);
```

```
    assertEquals("Asha", name);  
}
```

No database is involved. The repository pretends to have found the user.

5. Verification

Verification checks whether a dependency method was called.

```
verify(emailSender).sendWelcomeEmail(user);
```

Example:

```
@Test  
void sendsWelcomeEmailAfterRegistration() {  
    UserRepository repository = mock(UserRepository.class);  
    EmailSender emailSender = mock(EmailSender.class);  
  
    UserService service = new UserService(repository, emailSender);  
    User user = new User("maya@example.com");  
  
    service.register(user);  
  
    verify(repository).save(user);  
    verify(emailSender).sendWelcomeEmail(user);  
}
```

This checks behavior:

- Was the user saved?
- Was the welcome email sent?

You can also verify that something never happened:

```
verify(paymentGateway, never()).charge(anyDouble());
```

6. Test Doubles

A test double is any replacement object used during testing.

Common types:

- Dummy: passed around but not actually used
- Stub: returns predefined data
- Mock: verifies interactions
- Fake: simplified working implementation
- Spy: wraps a real object and watches calls

Mockito mostly helps with mocks, stubs, and spies.

7. Avoiding Over-Mocking

Over-mocking happens when a test replaces too many things with mocks.

Bad smell:

```
User user = mock(User.class);
Order order = mock(Order.class);
Address address = mock(Address.class);
```

Better:

```
User user = new User("Alex", "alex@example.com");
Order order = new Order(100.0);
```

Good rule:

Mock dependencies, not data.

Mock this:

```
UserRepository repository = mock(UserRepository.class);
PaymentGateway gateway = mock(PaymentGateway.class);
EmailSender emailSender = mock(EmailSender.class);
```

Usually do not mock this:

```
User user = mock(User.class);
Order order = mock(Order.class);
```

8. Mockito Annotations

Instead of manually creating mocks, many JUnit 5 tests use Mockito annotations.

```
import org.junit.jupiter.api.extension.ExtendWith;
import org.mockito.InjectMocks;
import org.mockito.Mock;
import org.mockito.junit.jupiter.MockitoExtension;

@ExtendWith(MockitoExtension.class)
class UserServiceTest {

    @Mock
    UserRepository userRepository;

    @Mock
    EmailService emailService;
```

```
@InjectMocks
UserService userService;
}
```

`@Mock` creates fake dependencies.

`@InjectMocks` creates the real class under test and injects the mocks into it.

Practice Exercises

Exercise 1: Mock a Repository

Create a `UserService` that depends on `UserRepository` .

Test that:

- `getUserName(1L)` returns the user's name
- `getUserName(99L)` returns `"Unknown"` when no user exists

Practice:

```
when(userRepository.findById(1L)).thenReturn(Optional.of(user));
```

Focus: `mock` , `when` , `thenReturn` , assertions.

Exercise 2: Verify a Method Was Called

Create a `RegistrationService` that depends on:

```
UserRepository
EmailService
```

When a user registers, the service should save the user and send a welcome email.

Test:

```
verify(userRepository).save(user);
verify(emailService).sendWelcomeEmail(user.getEmail());
```

Focus: `verify` .

Exercise 3: Test Failure Paths

Create a `PaymentService` that depends on `PaymentGateway` .

If the gateway returns `false` , the service should return:

```
PAYMENT_FAILED
```

Practice:

```
when(paymentGateway.charge(100.0)).thenReturn(false);
```

Test both success and failure.

Focus: testing multiple outcomes.

Exercise 4: Verify a Method Was Never Called

Create an `OrderService` .

If the order total is `0` or negative, it should not call the payment gateway.

Test:

```
verify(paymentGateway, never()).charge(anyDouble());
```

Focus: preventing unwanted behavior.

Exercise 5: Use Argument Matchers

Create a `NotificationService` that calls:

```
emailClient.send(String to, String subject, String body);
```

Use matchers:

```
verify(emailClient).send(
    eq("student@example.com"),
    eq("Welcome"),
    anyString()
);
```

Focus: `eq` , `anyString` , `any` , `anyLong` .

Exercise 6: Use ArgumentCaptor

Create an `InvoiceService` that sends an `Invoice` object to `InvoiceRepository` .

Use `ArgumentCaptor` to inspect what was saved:

```
ArgumentCaptor<Invoice> captor = ArgumentCaptor.forClass(Invoice.class);

verify(invoiceRepository).save(captor.capture());

Invoice savedInvoice = captor.getValue();
assertEquals(100.0, savedInvoice.getAmount());
```

Focus: checking complex objects passed to dependencies.

Exercise 7: Throw Exceptions From a Mock

Create a `FileUploadService` that depends on `StorageClient` .

Practice:

```
when(storageClient.upload(file)).thenThrow(new RuntimeException("Upload failed"));
```

Test that your service handles the exception properly.

Focus: testing error handling.

Exercise 8: Mock a Void Method

Create an `AuditService` with a dependency:

```
AuditLogger.log(String message)
```

Since `log` returns `void` , practice:

```
doThrow(new RuntimeException("Logger unavailable"))  
  .when(auditLogger)  
  .log(anyString());
```

Focus: `doThrow` , `doNothing` , and void method testing.

Exercise 9: Avoid Over-Mocking

Write two versions of a test for `OrderService` .

Bad version:

```
Order order = mock(Order.class);
```

Better version:

```
Order order = new Order(100.0);
```

Compare which test is easier to read.

Exercise 10: Use `@Mock` and `@InjectMocks`

Rewrite earlier tests using annotations:

```
@ExtendWith(MockitoExtension.class)  
class UserServiceTest {  
  
    @Mock  
    UserRepository userRepository;
```



```
@InjectMocks
UserService userService;
}
```

Focus: cleaner Mockito setup.

Lab: Testing a User Registration Workflow With Mockito

Goal

Build and test a small registration workflow using Mockito.

You will practice:

- Creating mocks
- Stubbing return values
- Verifying method calls
- Testing failure paths
- Using `never()`
- Using `ArgumentCaptor`
- Avoiding over-mocking

Scenario

You are building a user registration feature.

When a user registers:

1. Check if email already exists
2. If email exists, reject registration
3. If email does not exist, save the user
4. Send a welcome email
5. Write an audit log

Classes to Create

```
public class User {
    private Long id;
    private String name;
    private String email;

    public User(String name, String email) {
        this.name = name;
        this.email = email;
    }

    public Long getId() {
        return id;
    }
}
```

```
public String getName() {  
    return name;  
}  
  
public String getEmail() {  
    return email;  
}  
}
```

```
public interface UserRepository {  
    boolean existsByEmail(String email);  
    User save(User user);  
}
```

```
public interface EmailService {  
    void sendWelcomeEmail(String email);  
}
```

```
public interface AuditLogger {  
    void log(String message);  
}
```

```
public class RegistrationService {  
    private final UserRepository userRepository;  
    private final EmailService emailService;  
    private final AuditLogger auditLogger;  
  
    public RegistrationService(  
        UserRepository userRepository,  
        EmailService emailService,  
        AuditLogger auditLogger  
    ) {  
        this.userRepository = userRepository;  
        this.emailService = emailService;  
        this.auditLogger = auditLogger;  
    }  
  
    public String register(String name, String email) {  
        if (userRepository.existsByEmail(email)) {  
            auditLogger.log("Registration rejected for existing email: " + email);  
            return "EMAIL_ALREADY_EXISTS";  
        }  
  
        User user = new User(name, email);  
        userRepository.save(user);  
        emailService.sendWelcomeEmail(email);  
        auditLogger.log("User registered: " + email);  
    }  
}
```

```
        return "SUCCESS";
    }
}
```

Test Setup

Use JUnit 5 and Mockito.

```
import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.extension.ExtendWith;
import org.mockito.ArgumentCaptor;
import org.mockito.InjectMocks;
import org.mockito.Mock;
import org.mockito.junit.jupiter.MockitoExtension;

import static org.junit.jupiter.api.Assertions.assertEquals;
import static org.mockito.Mockito.any;
import static org.mockito.Mockito.anyString;
import static org.mockito.Mockito.doThrow;
import static org.mockito.Mockito.inOrder;
import static org.mockito.Mockito.never;
import static org.mockito.Mockito.verify;
import static org.mockito.Mockito.when;

@ExtendWith(MockitoExtension.class)
class RegistrationServiceTest {

    @Mock
    UserRepository userRepository;

    @Mock
    EmailService emailService;

    @Mock
    AuditLogger auditLogger;

    @InjectMocks
    RegistrationService registrationService;
}
```

Task 1: Successful Registration

Write a test where the email does not already exist.

Expected behavior:

- `register()` returns `"SUCCESS"`
- `userRepository.save()` is called
- `emailService.sendWelcomeEmail()` is called

- `auditLogger.log()` is called

Starter:

```
@Test
void registerReturnsSuccessWhenEmailDoesNotExist() {
    when(userRepository.existsByEmail("alex@example.com")).thenReturn(false);

    String result = registrationService.register("Alex", "alex@example.com");

    assertEquals("SUCCESS", result);

    verify(userRepository).save(any(User.class));
    verify(emailService).sendWelcomeEmail("alex@example.com");
    verify(auditLogger).log("User registered: alex@example.com");
}
```

Task 2: Email Already Exists

Write a test where the email already exists.

Expected behavior:

- `register()` returns `"EMAIL_ALREADY_EXISTS"`
- `userRepository.save()` is never called
- `emailService.sendWelcomeEmail()` is never called
- `auditLogger.log()` records rejection

Solution shape:

```
@Test
void registerRejectsExistingEmail() {
    when(userRepository.existsByEmail("alex@example.com")).thenReturn(true);

    String result = registrationService.register("Alex", "alex@example.com");

    assertEquals("EMAIL_ALREADY_EXISTS", result);

    verify(userRepository, never()).save(any(User.class));
    verify(emailService, never()).sendWelcomeEmail(anyString());
    verify(auditLogger).log("Registration rejected for existing email: alex@example.com");
}
```

Task 3: Capture the Saved User

Use `ArgumentCaptor` to inspect the `User` object passed into `save()`.

```
@Test
void registerSavesUserWithCorrectNameAndEmail() {
    when(userRepository.existsByEmail("maya@example.com")).thenReturn(false);
```

```

registrationService.register("Maya", "maya@example.com");

ArgumentCaptor<User> userCaptor = ArgumentCaptor.forClass(User.class);

verify(userRepository).save(userCaptor.capture());

User savedUser = userCaptor.getValue();

assertEquals("Maya", savedUser.getName());
assertEquals("maya@example.com", savedUser.getEmail());
}

```

Task 4: Verify Call Order

Check that the user is saved before the welcome email is sent.

```

@Test
void registerSavesUserBeforeSendingEmail() {
    when(userRepository.existsByEmail("sam@example.com")).thenReturn(false);

    registrationService.register("Sam", "sam@example.com");

    var inOrder = inOrder(userRepository, emailService);

    inOrder.verify(userRepository).save(any(User.class));
    inOrder.verify(emailService).sendWelcomeEmail("sam@example.com");
}

```

Task 5: Exception Handling Challenge

Modify `RegistrationService` so that if email sending fails, registration returns:

```
EMAIL_FAILED
```

Example test:

```

@Test
void registerReturnsEmailFailedWhenEmailServiceThrowsException() {
    when(userRepository.existsByEmail("nina@example.com")).thenReturn(false);

    doThrow(new RuntimeException("Email server down"))
        .when(emailService)
        .sendWelcomeEmail("nina@example.com");

    String result = registrationService.register("Nina", "nina@example.com");

    assertEquals("EMAIL_FAILED", result);

    verify(userRepository).save(any(User.class));
}

```

```
        verify(auditLogger).log("Email failed for: nina@example.com");  
    }
```

Update the service logic like this:

```
try {  
    emailService.sendWelcomeEmail(email);  
} catch (RuntimeException ex) {  
    auditLogger.log("Email failed for: " + email);  
    return "EMAIL_FAILED";  
}
```

Completion Checklist

You are done when you can confidently explain:

- Why `UserRepository` is mocked
- Why `User` is not mocked
- What `when(...).thenReturn(...)` does
- What `verify(...)` checks
- Why `never()` is useful
- When to use `ArgumentCaptor`
- How to mock void methods with `doThrow()`

The core Mockito rhythm is simple: arrange mock behavior, call the real service, assert the result, and verify important interactions.